Reporte

Equipo 16	
Nombre: Lesly Adamaris Cotlame Nieto Christopher Bethancourt Contreras Francisco Mora Ruiz	**Matrícula**: 2226958 2226948 2226912
Nombre del curso: Componentes de lógica de negocios.	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 3	**Actividad**: Diseño Arquitectural del Sistema
Fecha: 13/10/2026	

Bibliografia (Formato APA)

- Cohn, M. (2004). User stories applied: For agile software development. Addison-Wesley.
- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison
- Wesley.
- Fowler, M. (2003). Patterns of enterprise application architecture. Addison-Wesley.
- Martin, R. C. (2017). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.
- Beck, K. (2003). Test-driven development: By example. Addison-Wesley.
- Millett, S., & Tune, N. (2015). Patterns, principles, and practices of domain-driven design.

Objetivo:

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto

Procedimiento:

EVIDENCIA DE PRUEBAS AUTOMATIZADAS

Proyecto Eventos Nova

Este documento presenta la evidencia de la ejecución de las pruebas automatizadas del proyecto Eventos Nova. Las pruebas verifican los comportamientos programados en los escenarios que cubre la suite: reglas de negocio del modelo de dominio, coordinación de los casos de uso, límites transaccionales, reacciones a Domain Events y coherencia entre los diagramas, los documentos y el código.

1. Entorno de ejecución

Elemento	Valor
Sistema operativo	Linux (Ubuntu 24.04)
Editor	Terminal WezTerm (bash)
Lenguaje	Python 3.13.16
Marco de pruebas	unittest (biblioteca estándar de Python)

2. Procedimiento

Las pruebas se ejecutaron en una terminal bash de Linux (WezTerm), desde la carpeta raíz del proyecto EVENTOS_NOVA_U3_ARQUITECTURA_CLNEG, que contiene las carpetas src/ y tests/. El comando ejecutado fue:

```
python -m unittest discover -s tests -v
```

La opción discover localiza los módulos test_*.py de la carpeta tests y la opción -v muestra el nombre y el resultado de cada prueba. La salida completa se conserva en el archivo ejecucion-pruebas.txt.

3. Corrección realizada en una ejecución anterior

En una primera ejecución, la prueba test_documentos_sin_nombres_de_archivos_fuente, de test_coherencia_diagramas.py, falló porque la búsqueda de documentos Word también incluía los archivos de la carpeta tests/evidencias/. La evidencia de pruebas contiene referencias a extensiones como .docx, por lo que la propia prueba detectaba ese texto como si fuera un incumplimiento.

Para corregirlo, se ajustó la búsqueda en tests/test_coherencia_diagramas.py para excluir la carpeta tests/evidencias/ de la revisión documental, manteniendo la búsqueda recursiva de archivos Word en el resto del proyecto.

Después de aplicar el cambio, se ejecutó nuevamente la suite completa mediante el comando py -m unittest discover -s tests -v. Esa ejecución terminó con 70 pruebas aprobadas, sin fallos ni errores; la suite tenía entonces 70 pruebas y la sección 9 resume las ampliaciones posteriores. La corrección se limitó al alcance de la prueba de coherencia documental y no modificó la lógica de negocio del sistema.

4. Suites de pruebas

La carpeta tests del proyecto contiene seis módulos de prueba. La cantidad de pruebas de cada uno se obtuvo del registro de ejecución.

Archivo	Pruebas	Qué comprueba
`test_coherencia_diagramas.py`	14	Coherencia entre los entregables y el código: que los métodos, clases, estados y Domain Events del mapa de dominio existan en el código; que los documentos Word citen métodos existentes, usen una sola versión del modelo y no incluyan nombres de archivos fuente; que la matriz de trazabilidad no tenga caracteres invisibles ni identificadores mal formados; y que los diagramas de clases y de secuencia correspondan al código.
`test_correcciones_etapas_3_a_7.py`	12	Correcciones técnicas de las etapas 3 a 7: avisos de regularización, aplicación del cambio confirmado al Evento, condiciones del servicio adicional, sustitución tardía de proveedor, revisión administrativa de la cancelación, solicitudes en trámite con saldo vencido y rechazos devueltos como resultado.

Archivo	Pruebas	Qué comprueba
`test_decisiones_stakeholder.py`	12	Decisiones del stakeholder y correcciones finales: aplicación de lo confirmado antes del inicio (O-02), cierre con el motivo «Evento cancelado» de las solicitudes en trámite al cancelarse el Evento, operación confirmada que no se aplica al Evento cancelado y queda registrada en «Revisión», sustitución no autorizada que sigue en búsqueda y vencimiento del pago del servicio adicional a las 24 horas naturales.
`test_etapa9_casos_de_uso.py`	24	Recorrido de los casos de uso CU-01 a CU-05 con flujos principales, alternativos y de error: propuesta, contratación, cancelación y cierre del evento, cambios, servicios adicionales y sustitución de proveedores.
`test_etapa9_transacciones_y_reacciones.py`	10	Límites transaccionales (una raíz por transacción, reversión ante fallas, publicación de eventos después del commit), reacciones a Domain Events (reintentos e idempotencia) y dependencias entre las capas de dominio, aplicación e infraestructura.
`test_reglas_corregidas.py`	19	Reglas de negocio corregidas: cancelación por saldo vencido registrada por el Coordinador, anticipo no recibido, plazo de cinco días de los servicios adicionales y distinción entre el pago efectuado por el Cliente dentro del plazo y su registro y verificación posteriores por Administración (anticipo, cambios, servicios adicionales, saldo del contrato original y regularización).

5. Resultados

Indicador	Resultado
Pruebas ejecutadas	91
Pruebas aprobadas	91
Pruebas fallidas	0
Pruebas con error	0
Pruebas omitidas	0
Tiempo registrado	0.950 s
Resultado final	OK

El registro muestra las 91 pruebas con resultado «ok» y termina con «Ran 91 tests in 0.950s» y «OK». En unittest, «OK» sin anotaciones adicionales indica que no hubo fallas, errores ni pruebas omitidas.

6. Evidencia visual

La figura 1 es la captura real de la ventana de la terminal tomada al terminar la ejecución final, recortada al borde de la ventana y sin otras modificaciones. Muestra la parte final de la salida, el resumen «Ran 91 tests in 0.950s», el resultado «OK» y el indicador de la terminal; el comando ejecutado y el detalle de las 91 pruebas están en ejecucion-pruebas.txt, generado en la misma ejecución.



Figura 1. Ejecución de las pruebas automatizadas del proyecto Eventos Nova: 91 pruebas ejecutadas y resultado OK

7. Interpretación y limitaciones

El resultado acredita que los 91 escenarios programados en la suite se comportan como esperan sus aserciones en la versión ejecutada. No demuestra por sí solo que exista una aplicación gráfica completa, que funcionen integraciones externas reales ni que estén cubiertos todos los casos posibles.

Las pruebas usan los componentes del código base del proyecto. Los resultados no sustituyen la revisión de las demás etapas de la actividad ni implican que todas estén aprobadas.

8. Conclusión

La ejecución final de python -m unittest discover -s tests -v terminó con 91 pruebas ejecutadas, 91 aprobadas y resultado OK en 0.950 segundos. Esta evidencia respalda la validación del código para los escenarios cubiertos por los seis módulos de prueba y sirve como registro verificable de su ejecución.

9. Ejecución posterior a la revisión de consistencia

Después de la revisión de consistencia con decisiones del stakeholder se agregó el módulo test_decisiones_stakeholder.py, con 4 pruebas, y se actualizó la prueba de la observación O-02, que ahora comprueba que un cambio confirmado antes del inicio se aplica al Evento. La suite completa se ejecutó con python -m unittest discover -s tests -v en un entorno Linux con Python 3.13 y terminó con Ran 74 tests in 0.512s y OK: 74 pruebas ejecutadas, 74 aprobadas, 0 fallidas, 0 errores y 0 omitidas. La salida completa de esta ejecución reemplaza a la anterior en ejecucion-pruebas.txt. Las secciones 1 a 8 y la figura 1 se actualizaron con la ejecución final descrita al terminar esta sección.

En la corrección final se agregaron 3 pruebas al mismo módulo, que queda con 7, para comprobar que al cancelarse el Evento las solicitudes en trámite se cierran con el motivo «Evento cancelado» y las confirmadas no; también se ajustaron 2 pruebas existentes del módulo. La suite completa se volvió a ejecutar con python -m unittest discover -s tests -v en Linux con Python 3.13 y terminó con Ran 77 tests in 0.554s y OK: 77 pruebas ejecutadas, 77 aprobadas, 0 fallidas, 0 errores y 0 omitidas. Su salida completa se sustituyó por la de la ejecución siguiente.

En la corrección del caso de la operación confirmada que no se aplicó antes de la cancelación se agregaron 5 pruebas al mismo módulo, que queda con 12: 3 comprueban que la operación confirmada no se aplica al Evento cancelado, que la solicitud conserva su confirmación y que el registro queda en «Revisión» con el motivo, y 2 comprueban que la sustitución no autorizada sigue en búsqueda y que el pago del servicio adicional vence a las 24 horas naturales. La suite completa se ejecutó con python -m unittest discover -s tests -v en Linux con Python 3.13 y terminó con Ran 82 tests in 0.677s y OK: 82 pruebas ejecutadas, 82 aprobadas, 0 fallidas, 0 errores y 0 omitidas. Su salida completa se sustituyó por la de la ejecución final.

En la auditoría final del BPMN no se modificaron el código ni las pruebas; se actualizaron el BPMN, la matriz de trazabilidad y los documentos. La suite completa se volvió a ejecutar con python -m unittest discover -s tests -v en Linux (Ubuntu 24.04, terminal WezTerm) con Python 3.13.16 y terminó con Ran 82 tests in 0.982s y OK: 82 pruebas ejecutadas, 82 aprobadas, 0 fallidas, 0 errores y 0 omitidas. Su salida completa se sustituyó por la de la ejecución final.

En la corrección final de los plazos de pago se distinguió el pago efectuado por el Cliente de su registro y verificación por Administración. Se agregaron 9 pruebas a test_reglas_corregidas.py, que queda con 19, y se ajustaron 2 pruebas existentes del mismo módulo para que el cierre por anticipo no recibido ocurra después de las 24 horas naturales posteriores al contacto. La suite completa se ejecutó con python -m unittest discover -s tests -v en Linux (Ubuntu 24.04, terminal WezTerm) con Python 3.13.16 y terminó con Ran 91 tests in 0.624s y OK: 91 pruebas ejecutadas, 91 aprobadas, 0 fallidas, 0 errores y 0 omitidas.

En la auditoría de consistencia documental solo se corrigió en el código el texto del motivo de cierre por falta de pago de los cambios y los servicios adicionales, que

decía «Pago no verificado en 24 h naturales» y ahora dice «El Cliente no efectuó el pago en las 24 h naturales»; no se agregaron ni se modificaron pruebas. La suite completa se volvió a ejecutar con python -m unittest discover -s tests -v en Linux (Ubuntu 24.04, terminal WezTerm) con Python 3.13.16 y terminó con Ran 91 tests in 0.950s y OK: 91 pruebas ejecutadas, 91 aprobadas, 0 fallidas, 0 errores y 0 omitidas. La salida completa de esta ejecución es la que contiene ejecucion-pruebas.txt y la figura 1 la muestra.